LogiPlex Connector Implementation Guide
SailPoint IdentityIQ

Professional implementation document including architecture, configuration, split logic, and code reference

1. Document Purpose

This document provides a professional implementation guide for the LogiPlex Connector in SailPoint IdentityIQ. It explains the business problem, architecture, connector configuration, split logic, aggregation behavior, provisioning behavior, and the actual split rule implementation used to logically separate OpenLDAP / Active Directory entitlements into business-facing logical applications.

2. Problem Statement

Multiple business applications were using a single directory source to manage application-specific groups. During aggregation, all groups were imported into one SailPoint application, which made entitlement visibility, access requests, certifications, and ownership reviews difficult.

3. Objective

The objective is to logically separate entitlements into application-specific logical applications while still using a single physical source system. This provides clear application ownership, simpler access requests, and cleaner certification scope without duplicating the source connector.

4. Solution Components

The implementation uses the following components:

- Source Application – the real connector connected to OpenLDAP or Active Directory
- LogiPlex Connector – adapter connector used to create logical application separation
- Custom Object – stores application-to-OU mapping
- Split Rule – routes accounts and groups to the correct logical application
- Provisioning Rule – translates logical requests back to the source connector

5. Flow Explanation

Aggregation Flow

1. The source connector aggregates accounts and groups from the directory.
2. LogiPlex invokes the split rule for each object.
3. The split rule evaluates group DN / OU values against the Custom Object mapping.
4. Matching entitlements are cloned into their target logical applications.
5. The original source application object is retained as the base object.

Provisioning Flow
1. User requests access in a logical application.
2. SailPoint builds a provisioning plan.
3. The LogiPlex provisioning rule translates the logical request.
4. The request is sent to the underlying real connector.
5. The source directory is updated.

6. Custom Object Design

Custom Object Name:
Logiplex-openldap

Example mapping:
Finance Application = ou=finance,ou=groups,dc=example,dc=com
HR Application = ou=hr,ou=groups,dc=example,dc=com
Sales Application = ou=sales,ou=groups,dc=example,dc=com

The split rule reads this Custom Object and checks whether the incoming group DN contains one of the configured OU values.

7. Connector XML Configuration

The application XML must be updated to use the LogiPlex connector and include the required adapter entries.

Key attributes:
• connector = sailpoint.services.standard.connector.LogiPlexConnector
• logiPlexAggregationRule = ISS_Logiplex_Split_Rule
• logiPlexMasterConnector = sailpoint.connector.ADLDAPConnector
• logiPlexPrefix = Adaptive
• logiPlexProvisioningRule = ISS LogiPlex Provisioning Rule

8. Split Rule Design Overview

The split rule performs two major functions:

Account processing
• Reads the list of groups from the aggregated account object
• Resolves each group DN to a target logical application
• Creates cloned account ResourceObjects with only the groups belonging to that logical application

Group processing
• Reads the DN of the aggregated group object

• Resolves the group DN to a target logical application
• Creates a cloned group ResourceObject for the matching logical application

9. Split Rule Logic – Functional Walkthrough

A. resolveApplication(dn)
This helper method reads the Custom Object mapping and returns the matching logical application based on the incoming DN.

B. Account Object Handling
When objectType = account:
• Read all groups from the account
• Resolve each group to an application
• Build an application-to-groups map
• Clone the account once per logical application
• Keep only that application's groups in the clone

C. Group Object Handling
When objectType = group:
• Read the group DN from dn / distinguishedName / groups attribute
• Resolve the application
• Clone the group object into the matching logical application

D. Default Behavior
If the object type is unknown, return the original object without splitting.

10. Actual Split Rule Code

The following code is the implementation used for LogiPlex split processing:

```
import sailpoint.object.Custom;
import sailpoint.object.ResourceObject;
import sailpoint.tools.Util;
import java.util.Map;
import java.util.HashMap;
import java.util.ArrayList;
import java.util.List;

log.error("===== Logiplex Split Rule Started =====");

public List resolveApplication(String dn) {

  List appList = new ArrayList();
```

```
try {

 if (Util.isNullOrEmpty(dn)) {
  return appList;
 }

 Custom customObject =
  context.getObjectByName(Custom.class, "Logiplex-openldap");

 if (customObject == null) {

  log.error("Custom Object Logiplex-openldap not found");

  return appList;
 }

 Map configMap = customObject.getAttributes().getMap();

 for (Object obj : configMap.entrySet()) {

  Map.Entry entry = (Map.Entry) obj;

  String appName = (String) entry.getKey();
  String ouDN = String.valueOf(entry.getValue());

  log.error("Checking App = " + appName);
  log.error("Configured OU = " + ouDN);
  log.error("Incoming DN = " + dn);

  if (dn.toLowerCase().contains(ouDN.toLowerCase())) {

   appList.add(appName);

   log.error("Matched Application = " + appName);

   break;
  }
 }

} catch (Exception e) {

 log.error("Error in resolveApplication", e);

}
```

```
  return appList;
}

Map resultMap = new HashMap();

String applicationName = application.getName();

String objectType = object.getObjectType();

log.error("Object Type = " + objectType);
log.error("Object Attributes = " + object.getAttributes());

if ("account".equalsIgnoreCase(objectType)) {

  log.error("Processing Account Object");

  List groups = object.getStringList("groups");

  log.error("Groups from account = " + groups);

  Map appToGroupsMap = new HashMap();

  if (groups != null && !groups.isEmpty()) {

    for (Object groupObj : groups) {

      String groupDN = (String) groupObj;

      List appNames = resolveApplication(groupDN);

      for (Object appObj : appNames) {

        String appName = (String) appObj;

        List existingGroups = (List) appToGroupsMap.get(appName);

        if (existingGroups == null) {
          existingGroups = new ArrayList();
        }

        if (!existingGroups.contains(groupDN)) {
          existingGroups.add(groupDN);
        }
```

```
      appToGroupsMap.put(appName, existingGroups);
    }
   }
  }

  resultMap.put(applicationName, object);

  for (Object key : appToGroupsMap.keySet()) {

   String appName = (String) key;

   List appGroups = (List) appToGroupsMap.get(appName);

   ResourceObject cloneObject = object.deepCopy(context);

   cloneObject.put("groups", appGroups);
   cloneObject.put("description", "Split Account from OpenLDAP");

   resultMap.put(appName, cloneObject);

   log.error("Created Account Split App = " + appName + " Groups = " + appGroups);
  }

 } else if ("group".equalsIgnoreCase(objectType)) {

  log.error("Processing Group Object");

  String groupDN = null;

  Object dnObj = object.getAttribute("dn");

  if (dnObj == null) {
   dnObj = object.getAttribute("distinguishedName");
  }

  if (dnObj == null) {
   dnObj = object.getAttribute("groups");
  }

  if (dnObj != null) {
   groupDN = String.valueOf(dnObj);
  }
```

```
  log.error("Group DN = " + groupDN);

  resultMap.put(applicationName, object);

  if (Util.isNotNullOrEmpty(groupDN)) {

    List appNames = resolveApplication(groupDN);

    for (Object appObj : appNames) {

      String appName = (String) appObj;

      ResourceObject cloneObject = object.deepCopy(context);

      cloneObject.put("description", "Split Group from OpenLDAP");

      resultMap.put(appName, cloneObject);

      log.error("Created Group Split App = " + appName);
    }

  } else {

    log.error("Group DN not found on object");
  }

} else {

  log.error("Unknown Object Type");

  resultMap.put(applicationName, object);
}

log.error("Returning Objects = " + resultMap.keySet());

return resultMap;
```

11. Code Explanation by Section

Imports: Imports SailPoint classes such as Custom and ResourceObject, utility methods, and Java collection classes required for mapping and cloning logic.

resolveApplication(): Looks up the Custom Object named Logiplex-openldap and compares the incoming DN with each configured OU value. If a match is found, it returns the corresponding logical application.

Account split processing: For account objects, the rule reads all account groups, resolves each group to a logical application, groups them by application, and creates a cloned ResourceObject per application containing only that application's groups.

Group split processing: For group objects, the rule reads the DN attribute, resolves the logical application, clones the object, and inserts the clone into the result map for the matching application.

resultMap: The returned map contains the original application object and one or more cloned logical application objects. LogiPlex uses this map to build the final logical application views.

12. Validation Checklist

• Verify the Custom Object exists and contains correct OU mappings

• Verify the application XML references the correct split rule and provisioning rule

• Run account aggregation and confirm cloned accounts appear in the expected logical applications

• Run group aggregation and confirm group entitlements appear in the expected logical applications

• Test access request add/remove operations from each logical application

• Review logs for 'Matched Application' and 'Created Account Split App' / 'Created Group Split App' messages

13. Screenshots from Application XML

Figure 1. LogiPlex connector declaration in the application XML.

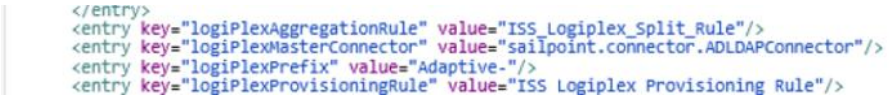

Figure 2. LogiPlex adapter entries including split rule, master connector, prefix, and provisioning rule.